LING 406: Term Project

Muhammad Kattom
University of Illinois Urbana-Champaign
mkatt9@illinois.edu

	Features	# of features	frequency or presence?	NB	ME	SVM
(1)	unigrams	16165	freq.	**78.7**	N/A	72.8
(2)	unigrams	"	pres.	81.0	80.4	**82.9**
(3)	unigrams+bigrams	32330	pres.	80.6	80.8	**82.7**
(4)	bigrams	16165	pres.	77.3	**77.4**	77.1
(5)	unigrams+POS	16695	pres.	81.5	80.4	**81.9**
(6)	adjectives	2633	pres.	77.0	**77.7**	75.1
(7)	top 2633 unigrams	2633	pres.	80.3	81.0	**81.4**
(8)	unigrams+position	22430	pres.	81.0	80.1	**81.6**

Figure 1: Pangs Benchmarks

Abstract

I visit bag-of-words pipelines for binary sentiment classification on a movie-review corpus. Using a logistic-regression classifier with a vectoriser achieves 85.5% accuracy while training in under a tenth of a second. I look at sevral approaches, talk of the results, and error.

1 Introduction

Sentiment analysis detects *good* or *bad* attitudes in text. Iy has uses in social-media monitoring and recommendation systems. This paper quantifies how far they can be pushed with different models and variable feature engineering.

2 Problem Definition

Given a movie review m_i, predict a label $l_i \in pos, neg$. Performance is measured by accuracy on an unseen 20 % test split

3 Previous Work

pang2004 benchmarked Naive Bayes, MaxEnt and SVMs on this corpus, reaching 82 % accuracy as seen in Figure 1. My goal is to maximize accuracy.

4 Approach

4.1 Dataset

I use the polarity corpus: 1000 positive and 1000 negative reviews. I use 80 % for training and 20 % for testing. Sample of 400.

4.2 Pre-processing and Tokenisation

Text is lowercased, punctuation is stripped, and tokens are extracted via sklearn's default whitespace tokenizer. Stop-words are removed in all experiments.

4.3 Baseline System

- **Model:** Naive Bayes
- **Features:** Bag-of-words unigram counts
- **Implementation:** `CountVectorizer` + `NB`

System	Accuracy	Training time (sec)
Baseline NB	0.818	0.01

Table 1: Baseline performance.

4.4 Algorithm Selection

With identical features, I train three additional models.

Model	Accuracy	Training time (sec)
Naive Bayes	0.818	0.01
Logistic Regression	**0.848**	0.30
Linear SVC	0.843	0.08
Random Forest	0.800	2.52

Table 2: Algorithm comparison results.

4.5 Feature Engineering

Fixing on Naive Bayes, I vary vectorization strategies.

Features	Accuracy	Training time (sec)	# Feat
Count (baseline)	0.818	0.01	35 850
TF–IDF	0.808	0.00	35 850
Count (min 5, max 0.7)	**0.835**	0.00	11 386
Count (binary)	0.833	0.00	35 850

Table 3: Feature-representation comparison.

4.6 Improved System

Combining the best algorithm and feature set:

System	Accuracy	Training time (sec)
LogReg + minmax	**0.855**	0.09

Table 4: Final system performance.

5 Error Analysis

Five random misclassifications are listed below. Some reasons for these errors seem to include negation scope, sarcasm, and mixed sentiment.

1. *True: neg* | *Pred: pos*
   "…you can't have any of this. It's all mine!…"

2. *True: neg* | *Pred: pos*
   "This movie … never really gets off the ground, despite some good performances…"

3. *True: pos* | *Pred: neg*
   "It was a rainy Friday ... when I persuaded a friend to see a ... MST3K …"

4. *True: pos* | *Pred: neg*
   "those men were talented artists , and their films reflect their genius . ? they are likely to be remembered for ..."

5. *True: pos* | *Pred: neg*
   "which means there are no smart network execs out there ... "

6 Future Work and Conclusions

Syntax-aware features can boost accuracy. I would also like to see how a deep learning model like neural networks would fare with this task.